



M363 Unit 9

UNDERGRADUATE COMPUTING

Software engineering with objects



From model to
implementation

Unit 9

This publication forms part of an Open University course M363 *Software engineering with objects*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc; Motif is a registered trademark of the Open Group; Rational Unified Process is a registered trademark of International Business Machines Corporation; Unified Modeling Language and UML are trademarks of Object Management Group, Inc. Design by Contract is a trademark of Interactive Software Engineering. Other product and company names may appear in the M363 course material. Rather than use a trademark symbol with every occurrence of a trademarked name, we use the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by SR Nova Pvt. Ltd, Bangalore, India.

Printed in the United Kingdom by Thanet Press Ltd, Margate.

ISBN 978 0 7492 1615 3

1.1



1	Introduction	5
2	From requirements to code	6
2.1	Design by Contract	6
2.2	Assertions	7
2.3	Executing an assertion using Java	8
2.4	Summary of section	18
3	Controlling inheritance	19
3.1	What is substitutability?	19
3.2	Inheritance and meaning	20
3.3	Summary of section	24
4	Software components	25
4.1	Specifying components and their interfaces	27
4.2	Summary of section	29
5	Summary	30
	Index	31

M363 COURSE TEAM

Robin Laney, Course Chair, Author and Academic Editor

Leonor Barroca, Author

Ralph Greenwell, Course Manager

Charles Haley, Author

Nigel Kermode, Critical Reader

Martin Shepperd, External Assessor, Brunel University

Richard Walker, Critical Reader

Andy Allum, Course Website Developer

Kim Dulson, Software Procurement

Phillip Howe, Media Assistant

Callum Lester, Software Developer

Tara Marshall, Print Procurement

Sandy Nicholson, Freelance Copy Editor

Andy Seddon, Media Project Manager

Lucinda Simpson, Editor

Sue Stavert, Technical Tester

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

Introduction

The overall aim of this unit is to provide you with the tools and background necessary to move from the abstract representation of a system as a UML model, to its concrete ('nuts and bolts') implementation and installation (deployment) ready for use. *Units 5–8* dealt almost exclusively with issues related to the analysis and design of software systems. In this unit, we will look at how to ensure that design requirements are reflected in the implemented code, looking in particular at how contracts specified during design may be implemented in Java. We will also look at how inheritance can affect the contracts. Finally, we introduce the notion of components, which can be used to increase reuse and to structure a system for deployment and maintenance reasons.

2

From requirements to code

In *Unit 6* you studied how, in an object-oriented system, pre- and postconditions can be related to a client–supplier contract between objects and how they can be usefully employed in the process known as Design by Contract (DbC).

You also saw that part of the value of DbC was that it allowed traceability from requirements through to code. In this section and Section 3, you will study how a contract, established in the requirements specification, can be translated into code.

This section will concentrate on the technicalities of tracking an established requirement through to code using DbC. To do this, you will study the encoding of requirements as assertions, and how they can be transformed into assertions about code. In the process, you will see how DbC can improve the documentation of classes.

2.1 Design by Contract

Whenever one object obtains a service from another object, some form of contract is involved. The contract may be implied or it may be made explicit, but either way the client object is expecting that under certain conditions, the supplier object will achieve a particular result. This *conditions* → *result* expectation is the contract. Preconditions describe the conditions that must be true before an operation can be used (before the client can request the service), and the postcondition describes the result of the operation (what the supplier is expected to do).

If the precondition is satisfied, then the client has reason to expect that the result will be as expected. However, if the precondition is not satisfied, then the outcome is not determined. The postcondition might be satisfied, or it might not. In addition, there is no guarantee that an error will be indicated, unless that guarantee is part of the contract.

The point to note here is that a contract is not something that may or may not exist; it is always present. For example, in Java no one should expect to be able to pick a method at random, call it using random parameters, and have something good happen. The client always has expectations of the supplier, and the supplier always has expectations of the client. There are always pre- and postconditions, whether or not they are written down. If software is to operate correctly under all conditions, the contracts must be respected.

Java offers three kinds of support for checking pre- and postconditions.

- ▶ The first is at compile time: statements are rejected unless the supplier object supports the operation concerned (requesting an operation the supplier does not provide is one example of failing to meet preconditions).
- ▶ The second is built-in error-checking at run-time: various kinds of error conditions (for example, use of null pointers or divide by zero) result in exceptions, which may help to prevent errors from propagating and causing catastrophic program failure or permanent data loss.
- ▶ The third is Java's assertion mechanism: a built-in mechanism for specifying conditions that must be true at a specific point in the code. If pre- and postconditions are known, it is probably worth using the assertion mechanism to check them, at least during system testing.

2.2 Assertions

In Java, an *assertion* is a statement of what must be true at a particular point in the execution of a method. A *precondition assertion* asserts what must be true at the method's start, and a *postcondition assertion* asserts what must be true at the finish. It is possible to have intermediate assertions.

An assertion is neutral about responsibilities – it only says what must be true at a particular point in the method's code if the method's execution is correct. The responsibility for ensuring the condition is met will lie elsewhere. An assertion must have no side effects; it must neither carry out any of the work of the program nor have any effect on the objects involved. It is simply true or false.

Connecting assertions with DbC, we see:

- ▶ if a precondition assertion is false, the client code has not met its responsibilities (to satisfy the precondition);
- ▶ if a postcondition assertion is false, the supplier code has not met its responsibilities (to deliver the postcondition, or do what is expected).

The class diagram in Figure 1 shows a simple bank system. We will look at inserting some assertions into the implementation of the withdrawal operation of *Account* to verify the pre- and postcondition.

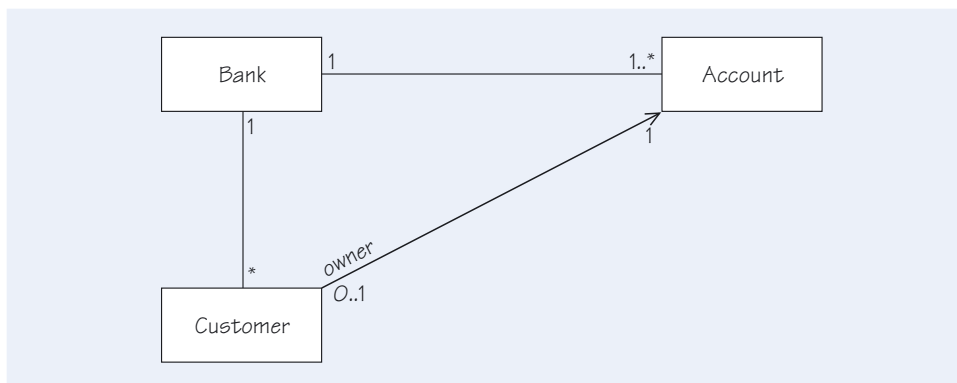


Figure 1 The business objects for the withdrawal operation

An example of a natural language description of the contract of a withdrawal operation is as follows.

If the withdrawal does not exceed the customer's overdraft limit, then decrease the customer's balance by the amount of the withdrawal.

In the same way as we did in Unit 6, we may identify pre- and postconditions from this natural language description as follows.

Precondition: the amount of the withdrawal must be positive, and the customer's balance (prior to the withdrawal) added to the customer's overdraft limit must be equal to or greater than the amount of the withdrawal.

Postcondition: the balance of the customer's account will have been decreased by the withdrawal amount.

To make an assertion is, according to the dictionary, to make a claim. In effect, an assertion is a statement that can be true or false in a similar way that a boolean expression in Java will be either `true` or `false`. In your translation of an assertion into code, you will draw heavily on the language of boolean expressions.

But precisely what sort of behaviour should an assertion produce when executed? Unlike a boolean expression, an assertion will not be used to choose a path through

subsequent code, as would be the case in the condition of an `if` statement. Rather, an assertion must be `true` at the point where it occurs *or else the program is in error*. A `true` assertion must therefore have no effect — allowing the program to continue as it would if the assertion was not there. A `false` assertion, on the other hand, indicates an error – a violation of a contract – and must be treated as an error. The actions that are taken when such an error occurs define an **assertion (handling) mechanism**.

SAQ 1

- (a) What is an assertion?
- (b) What is an assertion handling mechanism?
- (c) What does it mean to have an assertion mechanism in Java?

ANSWER.....

- (a) In general terms, an assertion is a claim that something is either true or false. In DbC, preconditions, postconditions and invariants are assertions. In Java, assertions are statements that evaluate to `true` or `false`. During execution an assertion that evaluates to `true` will have no effect; an assertion that evaluates to `false` indicates an error in the system.
- (b) An assertion handling mechanism (or assertion mechanism for short) is a programming construct, which, at run-time, allows an assertion to be executed. The main use of an assertion mechanism is when an assertion evaluates to `false`, in which case the assertion handling mechanism will, in some way, alert the user of the system to that fact.
- (c) To have an assertion mechanism in Java means being able to develop software in Java where the DbC conditions are checked at run-time.

2.3 Executing an assertion using Java

In Java, one puts assertions into the program using the `assert` statement. An `assert` statement has the form `assert boolean expression`. If the boolean expression evaluates to `true`, execution continues with the next statement. If the boolean expression evaluates to `false`, an `AssertionError` is thrown. **Assertion statements** are enabled or disabled at compile time. If they are disabled, the boolean expression is not evaluated and no exception is ever thrown. If they are enabled, the boolean expression is evaluated when encountered during the execution of a method, and the exception is thrown if the boolean expression evaluates to `false`. It might be appropriate to disable assertions when building 'final' versions of time-critical applications.

The general form for using Java assertions is shown in Figure 2.

```
method heading
  assert boolean expression for preconditions
method body
  assert boolean expression for postconditions
```

Figure 2 General form for assertion handling

SAQ 2

Suppose that we had included the following constructor in `BankAccount`.

```
public BankAccount(int anAmount, int anOverdraftLimit) {
    // to create a bank account given an initial balance,
    // and an overdraft limit
    assert anAmount >= 0 && anOverdraftLimit >= 0;

    // code for the constructor goes here
    assert getBalance() == anAmount &&
           getOverdraftLimit() == anOverdraftLimit;
}
```

Explain the meaning of the pre- and postcondition.

ANSWER.....

The precondition constrains the values of the arguments of the constructor to be correct: the precondition restricts the inputs to valid initial balances and valid overdraft limits.

The postcondition characterises what the initial state should be in terms of the values of the arguments. In setting the initial state, the constructor must return an object in a valid state; the object must satisfy the invariant of the class.

Figure 3 shows how Java assertions can be used to specify the precondition of the service to withdraw an amount from a bank account. As identified above in the natural language description for the pre- and postconditions, there are two data fields, `balance` and `overdraftLimit`, with associated 'getter' methods `getBalance()` and `getOverdraftLimit()` in the class containing the `withdraw` method. The `getOverdraftLimit` method is assumed to return how much a customer's account may be overdrawn.

```
public void withdraw(int anAmount) {
    // reduces the balance of an account provided that the
    // new balance does not exceed the overdraftLimit
    assert anAmount > 0 &&
           anAmount <= getBalance() + getOverdraftLimit();

    // the body of the method
}
```

Notice that we have avoided defining a 'money' type. Integers are used for simplicity.

Figure 3 A `withdraw` method checking precondition

Looking at the assertion, we see that we have simply encoded the natural language precondition as Java. It is phrased in terms of the 'getter' methods for the `balance` and `overdraftLimit` attributes.

Implementing the postcondition is more complex. The postcondition we want to verify is *the balance of the customer's account will have been decreased by the withdrawal amount*. To accomplish this, we must remember the value of the initial balance. However, we want to remember this value only if assertions are being used, because assertions can be turned off during compilation and we want to ensure that the presence of assertions in the code has no lingering effects on performance or behaviour if they are disabled. To remember the initial value, we use an 'inner class idiom' to capture the information we need later to do the postcondition check. Figure 4 shows one example of this, using the `Account` class.

In Java, it is legal to declare a class inside a method. These classes, called local inner classes, are visible only within the method. Methods in the inner class can use any variable or method in an enclosing method or class, with the restriction that variables used must be declared `final`. This is why in the example, `anAmount` is declared `final`.

```

public void withdraw(final int anAmount) {
    // reduces the balance of an account provided that the
    // new balance does not exceed the overdraftLimit
    class AssertionChecker {
        private int startBalance;
        AssertionChecker() { startBalance = getBalance(); }
    }
    boolean precondition() {
        return anAmount > 0 &&
            anAmount <= startBalance + getOverdraftLimit();
    }
    boolean postcondition() {
        return getBalance() == startBalance - anAmount;
    }
}
AssertionChecker assertCheck = null;
assert (assertCheck = new AssertionChecker()) != null &&
    assertCheck.precondition();

// the body of the method

assert assertCheck.postcondition();
}

```

Figure 4 A `withdraw` method checking pre- and postconditions

The inner class `AssertionChecker` stores a copy of the initial balance in a private attribute when an instance is created. It also contains two methods that use that attribute to implement the pre- and postconditions.

There are several things to note about the example in Figure 4.

- ▶ The declaration of the variable `assertCheck` must be outside of the assertion statement. The rules of Java do not permit declarations inside assertion statements. However, `assertCheck` is set to null instead of creating an instance, thereby avoiding doing any extra work if assertion statements are not enabled during compilation.
- ▶ If assertion statements are enabled at compile time, then an instance of `AssertionChecker` is allocated in the first `assert` statement, and the `precondition()` method is called. If the new fails or the `precondition` method returns false, the assertion fails.
- ▶ The second `assert` statement used the `postcondition()` method to check that the new balance is equal to the old balance less the amount withdrawn.
- ▶ The pre- and postcondition methods can implement conditions of whatever complexity is needed. As they are executed at run time only if assertions are enabled during compilation, the time penalty can be removed if needed when 'production' software is compiled.

In defining the pre- and postconditions in the inner class, we have not specified how its body should be implemented. All we require is that the precondition should evaluate to `true` before the body of the method is executed and the postcondition should evaluate to `true` just before the method returns. The method can be coded in any way which ensures that both pre- and postconditions evaluate to `true`, as we can then be sure it matches its specification and so is correct.

The absence of pre- or postconditions

Some methods, such as `getBalance()` and `getOverdraftLimit()`, may not have a precondition assertion. This does not mean that no precondition applies. It is simply a way of saying that the precondition is always `true`, which means that a client can make use of `getBalance()`, for example, without any restriction. The assumption being made in the `getBalance()` example is that if an instance of `Account` was successfully created, then the account must have a balance (a class invariant), and that `getBalance()` will always be able to return that balance.

Not all assertions are easy to express in a programming language

Representing every conceivable requirement as an executable assertion demands an assertion language of considerable expressive power. The Java mechanism presented in this unit does not have the power of natural language; consequently, some assertions cannot be expressed. For example, in Java it is very difficult to implement the precondition 'withdrawal from the account is permitted only if the account owner is living.'

However, even if it is not possible to express an assertion in code, it is good practice to include it informally. This can be done by including its natural language version (as would appear in the use case) as a comment at the appropriate place in the method. Although this does not allow a tool to test the assertion, it allows the contract for the service provided by the object to be documented for a programmer using the service.

The precondition of the `getBalance()` method (discussed above) is a good example. Writing an assertion of 'because this instance exists, the method can be executed' would become the uninteresting Java assertion `assert true;`. A comment explaining the rationale is far more useful.

SAQ 3

Why are some assertions not expressible in a programming language, and what should be done in this case?

ANSWER.....

The main reason is that the language in which assertions are written is not sufficiently expressive to enable the actual condition to be written. Also, some conditions depend on information the program does not have. In these cases, the assertion should be recorded as a comment, using natural language.

Inertial convention

It might be reasonable to say that all methods should include assertions in the postcondition that specifies which parts of the state are to remain unchanged. For example, the `withdraw()` method in Figure 4 could be amended to check that the overdraft limit has not changed during the execution of the method. Adding this check would give clients more certainty about what the method does.

However, it is tedious (and, therefore, error-prone) to state explicitly which components of an object's state remain unchanged during the execution of a method, particularly for objects that have many state components. Therefore, a useful convention is to assume that state components that are not explicitly described as having changed will remain unchanged after a method has been executed. Under this **inertial convention**, the postcondition in Figure 4 is sufficient.

'Inertial' is used to convey the physical notion that something will not change unless acted upon.

Class invariants and their relationship to pre- and postconditions

A `BankAccount` class might have an invariant consisting of an assertion stating that both the overdraft limit and the balance plus the overdraft limit must be greater than or equal to zero.

You saw in previous units that invariants on classes play an important role in capturing the constraints that must be imposed on the state of an object and on its associations with other objects. Another way of putting this is that, for any given class, legitimate objects of that class may be only in certain states. For example, a `BankAccount` object having a state where either the overdraft limit or a balance plus overdraft limit was negative, would not be a valid object of the class.

It is important to recognise that the state of an object may become invalid (an invariant does not hold) temporarily while it is being changed by one of its methods. For example, while a 'setter' method is executing, the statements in that method are running in sequence and the invariants may not be true again until the sequence finishes. This is not a problem for sequential systems, because if the method completes, the invariant again holds, and no other code can look into the object while the method is running. Unfortunately, this statement is not true in concurrent systems, where a method on some other thread might look at an object while the 'setter' method is in the middle of changing the object. In concurrent systems, the methods should protect the object by locking it to prevent other objects from discovering that the object is inconsistent. This is done in Java by using the `synchronized` keyword on all 'setter' methods that change the state of the object, and on all 'getter' methods for attributes that might be temporarily inconsistent because of the actions of 'setter' methods.

To summarise, **class invariants** are to be considered as part of both the pre- and the postconditions. Before a method can be executed, both the invariant and the precondition must hold. After the method has completed, the invariant and the postcondition must hold. This would seem to indicate that, before calling a method, the client must also check that the invariant is true. Fortunately, this is usually not necessary because if every method and constructor in the class ensures that the invariant holds on completion of the method, then the invariant will always be true before the next method is invoked. There is one exception: the invariant might not be true if a 'setter' method uses another internal method while the state is inconsistent (for example, in the middle of the sequence of statements). Methods used internally in this way may benefit from checking the class invariant in the precondition.

SAQ 4

What is one reason that 'getter' and 'setter' methods are useful in concurrent programs when using assertions to enforce contracts in Java?

ANSWER.....

The 'getter' and 'setter' methods provide good points to enforce synchronisation of access to data fields. Java supports locking of access to methods but not to attributes, so if there is a risk that a data field might be inconsistent for a time, `synchronized` 'getter' and 'setter' methods should be used.

Assertions in the constructor

The constructor(s) of a class instantiates (bring into existence) objects that are normally initialised to a state required by the client, and satisfy the invariant. A constructor, just like methods, may have pre- and postconditions.

From our discussion of invariants above, it is also clear that a major responsibility of the constructor is to create a 'valid' object; that is, one which satisfies its class invariant. This means that once an object is created, it will be in a valid state. To ensure that an object remains in a valid state after a method has been executed, both the method's postcondition and the invariant must hold.

Documentation

Executable assertions (written as code), as in the case of the `BankAccount` in Figure 4, provide the developer with exact detail of the contractual terms under which it can be used. In other words, assertions provide a powerful aid to documentation since they tell the developer what he or she needs to know in order to use a supplier object correctly.

However, if executable assertions are used, they should not be used as the *only* way of documenting a class, because they can be difficult to read. Although it is not strictly necessary, an important part of the documentation of a class is to give an informal description of what a method does in the comment that follows the heading of the method. Comments such as these complement executable assertions, and are helpful to the client developer when the executable assertions are complex.

Assertions in the supplier and in the client

The supplier code

You will recall from Section 2 of *Unit 6* that the precondition of a method is considered a benefit to the supplier object because it limits the inputs that the supplier needs to be prepared to handle. It should also be noted that the types of the formal arguments to the method (if any) also serve the same purpose. With these thoughts in mind, assume that you have been set the task of implementing the `BankAccount` class. What steps should you carry out?

First, you will need to decide how to implement the state, if that has not already been done. Second, you will focus on the pre- and postconditions of the methods. Lastly, you will look at the body of the methods.

SAQ 5

- (a) What are the essential steps in implementing a class specified in terms of assertions?
- (b) What are the essential steps in implementing the client code using a method specified in terms of assertions?

ANSWER.....

- (a) First, the state should be modelled. Second, the header should be defined for each method. Third, for each method, code should be included to ensure the postcondition and invariant are satisfied (remember that the precondition is a constraint on the client object).
 - (b) For the client, the precondition should be used as a test to ensure that any input to the service required is correct. This can be conveniently done using an if-then-else statement. The then branch could hold the message to call the service, with the else branch dealing with incorrect input.
-

Consider an example based on the `BankAccount` class. In this example, the decision about state is straightforward: in our bank currency units are not decimal fractions (that is, either pence or pounds, but not both), so the state of a `BankAccount` can be represented by two private integer data fields, `balance` and `overdraftLimit`. The 'getter' and 'setter' methods for these data fields can easily be supplied.

Regarding pre- and postconditions, consider the `deposit` method. We shall look at its precondition – that the amount being deposited is greater than zero. The type (`int`) of

the formal argument, `anAmount`, has done some of your implementation work for you, by eliminating some invalid input – in this case, all input that is not an integer.

Finally, you implement the body by updating the balance in such a way that the postcondition *and* invariant hold after the update; that is:

- ▶ `getBalance()` should return the new balance (as characterised by the postcondition shown in Figure 5);
- ▶ `getOverdraftLimit()` should remain unchanged;
- ▶ the final state should be consistent with the invariant.

Code that satisfies these constraints is shown in Figure 5.

```
public void deposit(int anAmount) {
    // a method that credits the balance with 'anAmount'
    assert anAmount > 0; // precondition
    setBalance(getBalance() + anAmount);
    // postcondition: check that the
    // new balance == old balance + anAmount
}
```

Figure 5 An implementation of the `deposit` method without the postcondition

SAQ 6

Implement the postcondition of the `deposit` method in Figure 5.

ANSWER.....

The following method uses the ‘inner class idiom’, shown previously, to implement the postcondition.

```
public void deposit(final int anAmount) {
    // a method that credits the balance with 'anAmount'

    // define the assertion checking class
    class AssertionChecker {
        private int startBalance;
        AssertionChecker() { startBalance = getBalance(); }
        boolean precondition() {
            return anAmount > 0;
        }
        boolean postcondition() {
            return getBalance() == startBalance + anAmount;
        }
    }

    // check the precondition
    AssertionChecker assertCheck = null;
    assert (assertCheck = new AssertionChecker()) != null &&
        assertCheck.precondition();

    // the body of the method
    setBalance(getBalance() + anAmount);

    // check the postcondition
    assert assertCheck.postcondition();
}
```

Figure 6 An implementation of the `deposit` method with the postcondition

The client code

When invoking a method on a `BankAccount` object, the client benefits by obtaining the service specified by the method's postcondition. The client's obligation is to guarantee the precondition of the method before invoking it. Let us see how you would fulfil this obligation in using the `withdraw` method on a `BankAccount` object.

First, we assume that you already know what `withdraw` will do for you (as specified in its postcondition).

You need to establish exactly the requirements you must fulfil in order to invoke `withdraw` correctly. These involve ensuring that:

- ▶ you provide the correct type of actual argument (or arguments) to the method;
- ▶ you meet the precondition.

In the case of `withdraw`, the type of the argument is `int` and the precondition states that the integer you provide must be positive and not exceed the existing balance plus the overdraft limit. Suppose that the `BankAccount` object is called `acct1`, the client code must be similar to that shown in Figure 7 (`anAmount` is assumed to be an integer variable that has already been initialised to some value).

```
if (anAmount > 0 &&
    anAmount <= acct1.getBalance() + acct1.getOverdraftLimit())
    acct1.withdraw(anAmount);
else
    // do whatever needs to be done with
    // other values of 'anAmount'
```

Figure 7 Client code that uses the `withdraw` method

It might come as a surprise that the client must check a condition that is essentially the precondition of the method. Is this unnecessary duplication? The answer is both 'yes' and 'no', and hinges on understanding the following three points.

First, the condition in the `if` statement of Figure 7 and the precondition of `withdraw` serve very different purposes. In the `if` statement the condition is required because different actions (one being the use of `withdraw`) are required for dealing with different values of `anAmount`. These actions will be subject to the client's own postcondition (remember the code in Figure 7 will be part of another method, most probably in another class). In the precondition of `withdraw`, the identical condition is *not* there to decide which of two possible strategies are needed to deal with different values of `anAmount`, but to assert that the program *is in error* (that is, there is a bug needing to be fixed) if an attempt is made to use `withdraw` with invalid input values.

Second, if assertions are executable, then when the system is being tested, the precondition check is made to ensure that the client code is correct and that the client has only called `withdraw` with appropriate values of `anAmount`. The precondition check is made only to ensure that the client is not violating the contract.

Third, having tested the program and found, by suitable testing strategies, that the precondition has not been violated, it can be assumed that the client code is correct. Assertions can then be switched off in the operational system, but the `if` in Figure 7 must remain.

Thus when testing the system, a check in both the client and the supplier does not constitute duplication, because the precondition in the supplier is acting as a monitor on the client.

The above example and discussion assumes that either the system is not concurrent, or the client code operation is protected from multiple execution (synchronized in Java). If neither condition is true, then the `BankAccount` object must be synchronized, the checks must be inside the `withdraw` method, and the contract for the `withdraw` method must not assume success but instead return success or failure. Otherwise, if two people simultaneously withdraw money, it is possible that one operation would violate the precondition because the other `withdraw` intervened between the first's check of the precondition and the invocation of the `withdraw` method.

When to use assertions

A question that naturally arises is, 'What happens when an assertion fails to be satisfied during execution?' We shall look at how assertions can be monitored at run-time later, but first we need to make it quite clear what assertions are *not* to be used for.

Assertions are not to be used to describe *expected cases* that call for special treatment. For example, if you know that a method normally should do a particular task but there is a certain case where it needs to take a different approach, then you should use standard conditional control structures to describe it. Assertions can be turned off, so they must not have side effects that the program depends upon.

Assertions are used for a different purpose: describing the conditions under which software elements will and will not work (preconditions) and the conditions that they will achieve (postconditions). If the supplier and its clients are correct, that is, keep to their contract, assertions should always evaluate to `true`. In other words, an assertion violation is an indication of a defect in the software.

- ▶ A precondition violation indicates a defect in the client.
- ▶ A postcondition violation indicates a defect in the supplier.

The use of precisely defined contracts clearly assigns all responsibilities to either the client or the supplier. Any problem that might occur at run-time is the responsibility of either the client or the supplier, *but not both*. The stronger the precondition, that is, the more restrictive it is, the higher the burden on the client (there are more conditions to be satisfied before the client is allowed to invoke the supplier) but the easier it makes the task of the supplier. In practice, it has been found that designs that concentrate on doing a well defined job in order to do it well are more successful than those that try to handle many cases. In terms of preconditions, this implies that stronger preconditions are better than weak ones.

This approach is often criticised because it seems to force every client to check that a precondition is satisfied before invoking the supplier, resulting in unnecessary work. However, the presence of a precondition does not necessarily mean that every invocation must verify the precondition. A precondition means simply that the client must guarantee the condition. This is *not* the same as testing for the condition as in the following code fragment:

```
if (precondition) {
    // invoke supplier
}
else {
    // do something special
}
```


If the context of the invocation implies that the precondition holds, such a test will not be needed. For example, suppose that an implementation of a container class deals with containers of a fixed upper size. There is a maximum number of elements that can be added to a container and there is always a problem of what to do when the container is full. However, if the client has just removed an item from the container, it is possible (at least in a sequential system) to add another element without checking for fullness.

Of course, there is a trade-off here. The precondition must be checked *somewhere*, unless we are willing not to check it and, as a consequence, not to check that the program is executing correctly. The trade-off involves where, and how frequently, to check the precondition in order to be confident that the program is behaving correctly.

Exercise 1

The `BankAccount` class is to be extended to include a `transfer` method of the following form:

```
public void transfer(int anAmount, BankAccount anAccount) {
    // transfers 'anAmount' from the current
    // account to 'anAccount'
}
```

- (a) Give suitable pre- and postconditions for this `transfer` method.
- (b) Implement the `transfer` method.

Solution

- (a) Using the inertial convention, the method will have the following pre- and postconditions:

Preconditions (all must be true):

```
1  anAccount != null
2  anAmount > 0
3  anAmount <= getBalance() + getOverdraftLimit();
```

Postconditions (all must be true):

```
1  getBalance() == previousBalance - anAmount
2  anAccount.getBalance() == previousBalance + anAmount;
```

- (b) A possible implementation is shown below:



```

public void transfer (final int anAmount,
                     final BankAccount anAccount) {
    // transfers 'anAmount' from the current account
    // to 'anAccount'

    // define the assertion checking class
    class AssertionChecker {
        private int startBalance;
        private int otherAccountStartBalance;
        AssertionChecker() {
            assert anAccount != null; // we will use it below!
            startBalance = getBalance();
            otherAccountStartBalance = anAccount.getBalance();
        }
        boolean precondition() {
            // anAccount was checked in the constructor
            return anAmount > 0 &&
                anAmount <= getBalance() + getOverdraftLimit();
        }
        boolean postcondition() {
            return getBalance() == startBalance - anAmount &&
                anAccount.getBalance() ==
                otherAccountStartBalance + anAmount;
        }
    }
    // check the precondition
    AssertionChecker assertCheck = null;
    assert (assertCheck = new AssertionChecker()) != null &&
        assertCheck.precondition();

    // the body of the method
    setBalance(getBalance() - anAmount);
    anAccount.setBalance(anAccount.getBalance() + anAmount);

    // check the postcondition
    assert assertCheck.postcondition();
}

```

Figure 8 An implementation of the `deposit` method

2.4 Summary of section

In this section, you have:

- ▶ learnt that a conditional statement determines which execution path is to be used, whereas an assertion states what must be true at that point in a program otherwise the program is in error;
- ▶ examined the relationship between invariants, and pre- and postconditions;
- ▶ used assertions to express the contract between the components of a simple application;
- ▶ learnt to incorporate Java assertions in a way that captures the specification of its methods and supports the notion of DbC;
- ▶ learnt how to implement both the supplier and client codes in accordance with a simple contract.

3

Controlling inheritance

In Section 2, you studied how an assertion mechanism can be used to apply the notion of a contract in the development of an object-oriented system. This is the basis of DbC. You also studied how assertions serve to document classes in terms of their contractual obligations, thus facilitating the correct use and reuse of such classes. At another level, if the assertions are executable, they serve to facilitate testing to determine whether a program is correct in terms of the specified contracts. The discussion in Section 2 was phrased in terms of a ‘client–supplier contract’; that is, the contract that exists between classes that are related to each other via *association*. However, the notion of a contract is also useful in guaranteeing that substitutability makes sense, that is, ensuring that a subclass honours the contract of its superclass.

3.1 What is substitutability?

The first condition for an object to be substitutable for another object is that the type of the first must either be the same as the second, or a subtype of the second. However, even though this condition is necessary, it is not sufficient. For example, consider a class SC, which is a subclass of class C. To be substitutable, objects of subclass SC must also *fulfil the same contract* as objects of the superclass C. It must always be possible for an object of subclass SC to stand in for an object of superclass C, without the original contract with the client being broken.

If a subclass has precisely the same precondition and postcondition as the superclass then the contract will be honoured, assuming that the conditions are completed. But what if the subclass *modifies* the precondition or the postcondition?

Here is a homely example of things going right (Fig. 9), and things going wrong. Assume that a client wants to buy 10 red roses. The client contacts FlowersRUS, an instance of the class of FlowerSupplier. FlowersRUS forwards the order to RosesLtd, an instance of RoseSupplier, which is a subclass of FlowerSupplier. RosesLtd offers that, given £5 (the precondition), it will deliver the roses (the postcondition).

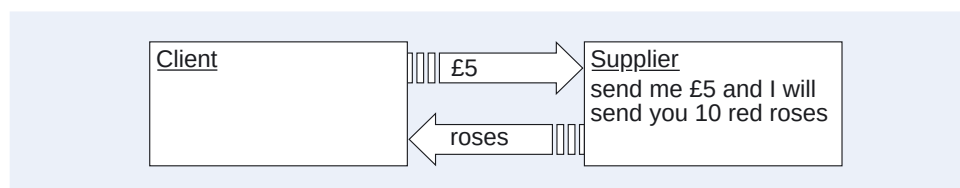


Figure 9 An example where both the client and supplier are happy

But what if RosesLtd is out of roses, and FlowersRUS forwards the order to a different supplier: RedFlowersInc, an instance of the class FlowerSupplier? As Figure 10 illustrates, although the client is still entitled to expect the original terms to hold – roses for £5 – the new supplier is not of the same subclass as RosesLtd, and may have a different postcondition. RedFlowersInc may deliver something other than what is expected.

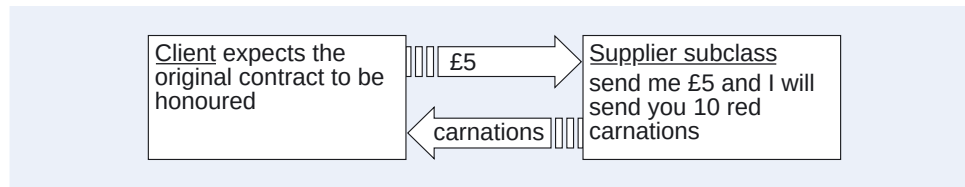


Figure 10 How changes to a postcondition can impact the client

Or, it could be that RedFlowersInc have different preconditions (see Fig. 11), and may demand something more than what is expected.

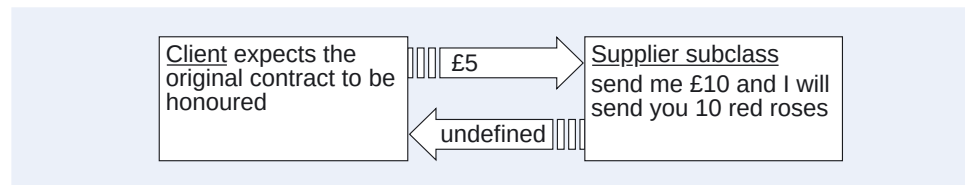


Figure 11 How changes to a precondition can impact the client

Either way, the contract with the client has been broken, and disappointment will result. Of course if the client got 20 red roses, or the price demanded fell to £2, the client would be contented, because the contract would be more than fulfilled.

This example illustrates that in a contract, the supplier cannot legitimately decide to demand *more* than originally promised, but it is at liberty to demand *less*. The supplier cannot legitimately decide to deliver less than originally promised, but it is at liberty to deliver *more*.

Under the rules of the Java language, an instance of a subclass is presumed to be substitutable for an instance of the superclass. As we said above, and as was illustrated by the flowers example, the class relationship is not sufficient. To be substitutable, the class and its subclasses must honour the same contracts. This means:

- ▶ preconditions should be the same, or demand less;
- ▶ postconditions should be the same, or deliver more.

Otherwise the original contract with clients will not be honoured.

3.2 Inheritance and meaning

You already know that it is possible to redefine inherited operations, and this raises the question of whether a proposed redefinition of a method makes design sense. For example, in building the class `Employee` as a subclass of `Person`, would redefining a method that computes the age of the `Person` to one that computes the number of years in employment instead, make sense, under the theory that the subclass is computing the 'age' of the person at the company? Probably not. In this section we shall examine this question in more detail and you will see how the assertions in DbC can be used to ensure that the behaviour of inherited operations 'makes sense'.

Inheritance without redefinition of methods

We will use another banking example to illustrate the subclassing contract. This time we assume the existence of two classes that are related via generalisation:

`SavingsAccount` (the superclass) and its specialised version

`SpecialSavingsAccount` (the subclass). The class `SavingsAccount` will have

methods similar to those of `BankAccount` (see Section 2); for example, `getBalance`, `setBalance`, `withdraw`, and `deposit`, but also others that are appropriate for savings

accounts. We shall be particularly concerned with the method `addInterest` from `SavingsAccount`, shown in Figure 12, that is assumed to be inherited by `SpecialSavingsAccount` without redefinition.

```
public void addInterest () {
    // adds interest to balance
}
```

Figure 12 The `addInterest` method

Since `SavingsAccount` and `SpecialSavingsAccount` are related via generalisation, it means that an object of class `SpecialSavingsAccount` is an object of class `SavingsAccount`. What this means is that all `SpecialSavingsAccount` objects must respond to the same methods as `SavingsAccount` objects, but does not preclude `SpecialSavingsAccount` objects responding to methods additional to those found in `SavingsAccount`.

In contract terms, this means that the methods that `SpecialSavingsAccount` inherits from `SavingsAccount` have the same contracts as those in `SavingsAccount`. Inheritance without redefinition guarantees that this will be the case. For example, since the `addInterest` method is the same for both `SavingsAccount` and `SpecialSavingsAccount`, it will implement the same contract when applied to an object of either class. The example given in Figure 13 illustrates this.

```
SavingsAccount sA = null;
SpecialSavingsAccount ssA = new SpecialSavingsAccount(...);
sA = ssA;
// later in the code
sA.addInterest(); // 1
```

Figure 13 Code illustrating subcontracting

The line marked `// 1` shows client code sending the `addInterest` message to an object referenced by `sA` (the supplier) a variable of type `SavingsAccount`. In this instance, the client expects the contract specified by `addInterest` as defined in `SavingsAccount` (see Figure 12) to be carried out on a `SavingsAccount` object. The client may not know that `sA` actually references an object of type `SpecialSavingsAccount`, via the mechanism of substitutability; but this causes no problem because the `addInterest` method is the same for both objects and hence will fulfil the expected contract.

SAQ 7

- What is the important characteristic of the subtyping relationship?
- Explain how inheritance without redefinition of inherited methods supports specialisation.
- Explain the meaning of each line of code in Figure 13 given that `addInterest` is not redefined in `SpecialSavingsAccount`.

ANSWER.....

- The subtyping relationship allows the subclass to be substituted for the class.
- Inheritance without redefinition does not prevent the addition of methods to the subclass. Specialisation can therefore be established through any added methods.

(c) The code is reproduced here for convenience:

```
1  SavingsAccount sA = null;
2  SpecialSavingsAccount ssA=new SpecialSavingsAccount(...);
3  sA = ssA;
4  sA.addInterest();
```

Line 1 assigns null to `sA`, which is a reference to a `SavingsAccount` object.

Line 2 creates a new `SpecialSavingsAccount` object and assigns it to `ssA`.

Line 3 assigns the `SpecialSavingsAccount` object referenced by `ssA` to the variable `sA` of type `SavingsAccount`, which is allowed under substitutability.

Line 4 sends the `addInterest` message to the `SpecialSavingsAccount` object referenced by `sA`. Through dynamic binding, this will cause the `addInterest` method of `ssA` to be invoked.

Inheritance with redefinition

The situation is more problematic in the case where an inherited method can be redefined.

The problem, in contract terms, can again be illustrated using the example shown in Figure 13, but using the `addInterest` method in Figure 12. In this case we assume that the `addInterest` method inherited by `SpecialSavingsAccount` has been redefined, for example, by paying a higher interest rate only if the balance exceeds some threshold.

For ease of reference, let us call this redefined method `addInterestssA`. As before, the target of the call in line // 1 will, through substitutability, be an object of type `SpecialSavingsAccount` and not `SavingsAccount`. However this time, dynamic binding will also ensure that it is the redefined `addInterestssA` that will be invoked and not the original `addInterest`. In other words, the client code's intended contract with `addInterest` from `SavingsAccount` is actually being subcontracted to (implemented by) `addInterest` redefined in `SpecialSavingsAccount`.

The important question that arises now is: what guarantee is there that the intended contract with `addInterest` will be fulfilled by `addInterestssA`? If the implementer of `addInterestssA` assumed that the method would not be called unless the balance was above the threshold (the precondition is `getBalance > threshold`), then there is a mismatch between the contract that the client expects and contract the supplier implements. The system will not function properly, even though visual inspection indicates that the types are correct and that the contracts should be respected.

A redefined method will not violate the contract of its parent method if:

- ▶ its precondition is the same or weaker (this allows more input);
- ▶ its postcondition is the same or stronger (this provides a 'better' service);
- ▶ the invariant of the redefining class is the same or stronger.

Intuitively, this makes sense. The client, in dealing with the original supplier, needs to provide input within a certain range. Consequently, the ability of the subcontractor to accept a wider range does not affect the client. Similarly, the client is happy to receive a response from the original supplier within a certain range. Therefore, if the subcontractor guarantees to provide a response within this range, an even smaller range will not affect the client. The `addInterestssA` example above violates the first rule, because it tightened the precondition.

This discussion may be summarised as follows.

- 1 The nature of the type–subtype relationship guarantees that if a client enters into a contract with a particular type, the contract must always be guaranteed if the task is subcontracted (via substitutability) to one of its subtypes.
- 2 Inheritance without the facility to redefine inherited methods guarantees that subclasses behave as true subtypes of their parent classes.
- 3 Inheritance with redefinition will only guarantee that subcontracting fulfils the original contract if the precondition of the method being redefined in the subclass is the same as, or weaker than, the corresponding precondition in the original method, and if the postcondition of the method in the subclass is the same as, or stronger than, the corresponding postcondition in the original method.

SAQ 8

- (a) Suppose that `op1` overrides `op2`. What are the implications in terms of input and output if:
- `op1` has a weaker precondition than `op2`;
 - `op1` has a stronger postcondition than `op2`.
- (b) What does it mean if `op1` has both a weaker precondition *and* a stronger postcondition than `op2`?

ANSWER.....

- (a) If method `op1` has a weaker precondition than `op2`, then `op1` will accept all input that is accepted by `op2`.
If method `op1` has a stronger postcondition than `op2`, then the output of `op1` will also satisfy the postcondition for `op2`.
- (b) It means that `op1` fulfils the same contract as `op2`.

Exercise 2

Given the specification of `addInterest` in Figure 12, explain, using the call to `addInterest` on line // 1 of Figure 13 as an example, what problems would be caused if the specification of `addInterest` in `SpecialSavingsAccount` were redefined as follows:

```
public void addInterest() {
    // adds interest to balance
    // precondition: getBalance() > 1000;
    // postcondition:
    // getBalance() = (previous balance) + (calculated interest)
    // rest of state stays the same
}
```

Solution.....

The precondition for `addInterest` in the `SavingsAccount` was true, and so did not require the client to check that its precondition was satisfied before sending the `addInterest` message in line // 1 of Figure 13. If however, the precondition of `addInterest` in `SpecialSavingsAccount` were changed as indicated (this is a strengthening of the precondition) it means that the call on line // 1 might well fail because the client has not been designed to meet the contract implied by the new precondition. For example, if the client invoked `addInterest` on an account having a



balance of less than a 1000, the call would fail. This illustrates why preconditions should only be left the same or weakened in redefined methods.

3.3 Summary of section

In this section, you have learnt:

- ▶ that inheritance without redefinition guarantees that a subclass will honour its parent's contracts, and poses no threat in a system that supports substitutability;
- ▶ how redefined inherited methods can undermine the subtyping relationship and cause problems in a language that supports substitutability;
- ▶ that inheriting assertions and being able to modify them in certain well defined ways enable inherited methods to be redefined in a way that does not pose any problems;
- ▶ that we need to consider whether one class inheriting from another makes conceptual sense.

4

Software components

Software components are units of code that can be flexibly plugged together to build software applications. The notion of a software component is similar to electrical engineering practice, where standard components can be assembled to build complex components and systems. Software components are particularly important in today's enterprise application development: the ability to build software by connecting, modifying, replacing or reusing software components gives rise to flexible systems which better meet the volatile requirements of rapidly changing organisations.

Software components, or simply *components*, are units of software structured according to some specific object-oriented principles. These are:

- ▶ a component encapsulates data and functions;
- ▶ a component comes with a specification that clearly separates the component's interface from its implementation;
- ▶ a component is known to client components only through its specified interface.

Although components need to adhere to these object-oriented principles, they do not have to be implemented using object technology. However, object technology is particularly suited to component development, and most components today are built with this technology.

SAQ 9

Name another concept that uses principles similar to those given above.

ANSWER.....

From the above principles, a component resembles the concept of an object very closely. Indeed, at a programming level, a component may well be implemented as a class instance, although it is more likely to be represented as a set of related class instances and perceived as a well defined subsystem within an application.

History of software components

The idea of software components goes back a long way. In 1968, in a paper given at the NATO conference that launched software engineering, Doug McIlroy recognised that libraries of utility programs to do tasks such as sorting, and libraries of mathematical routines, were very useful. Libraries saved the effort of writing programs from scratch every time, and also provided programs that had been thoroughly tested both in their original development and through repeated use by others. He wondered whether this idea could be spread to all software so that elements of software identified as generally useful could be stored in libraries and reused as needed.

At that time the experience in the industry was that many software requirements were not new, and had been developed before in a slightly different context. This seemed wasteful, but the best that could be done was to hire people who had done it before and pay them to do it again. For many outside the software industry, this seemed rather profligate, particularly when software engineers did not want to write the same software again and again, and would refuse to do so.

Software engineers took their inspiration from how other industries worked with components. It seemed that what was needed was a comprehensive set of software modules or components from which all software could be built. The components would need to be precisely specified in terms of their interfaces and the functions carried out, and some method of connecting them would be needed. By the late 1970s, a number of solutions had been developed by research communities, under the general title Module Interconnection Languages (MILs), following a ground-breaking paper by DeRemer and Kron (1976).

Generally, software components are plugged together to generate larger components or subsystems. In order to do so, components have a number of points of interconnection: each point is represented by an interface and a set of related operations corresponding to some specific service. In general, an interface is a description of available operations, such as the public methods provided by a class. Each component must distinguish between two types of interface: those *provided* for other components to use, and those *required* from other components. A *required interface* defines the operations that the component will need to request from other components; a *provided interface* defines the operations that the component offers to other components. Java provides a construct to define interfaces, called (naturally enough) an *interface*. Interfaces permit the designer to specify the types and signatures of the methods that an object that implements the interface will provide or need at run time. We will see more about interfaces below and in Unit 10.

Components are represented in UML as stereotyped boxes, where the stereotype can be graphical or textual. Figure 14 illustrates a component in the two notations. The figure also indicates the notation for required and provided interfaces: *IRequired* is a required interface, while *IProvided* is a provided interface.

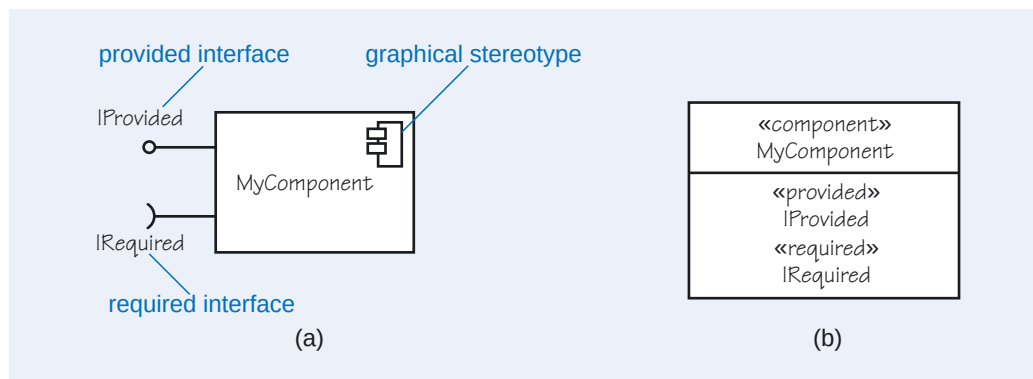


Figure 14 UML notation for components

Components are put together through their provided and required interfaces: a required interface of one component 'plugs' into a provided interface of another component. Component interconnection can be captured in UML as illustrated in Figure 15, where a component *Account* makes use of some security services provided by a *Security* component.

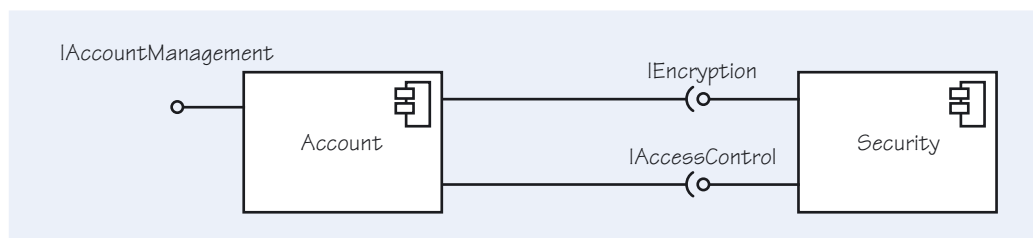


Figure 15 Interconnecting components

4.1 Specifying components and their interfaces

A component is a container of software artefacts, and its specification is in terms of those artefacts. Here we focus on components that are built using object technology, although components are not restricted to this technology.

In UML an interface is represented as a box, as shown in Figure 16. The stereotype *«interface»* is used to distinguish it from a class definition. This is similar to a class icon but the attribute compartment is always empty, as interfaces cannot declare attributes. Operations are declared though their complete signature, that is, the operation name with its parameters and optional returned values. An interface is used to define a cohesive set of behaviours, irrespective of how they are going to be implemented.

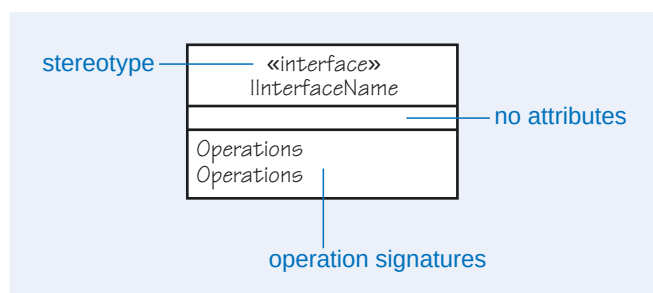


Figure 16 Specifying interfaces in UML

In Java, you cannot declare an instance of an interface. You must instead create a class that realises (implements) the interface. Such a class must contain operations (methods) that exactly match the signatures of the operations in the interface specification. In other words, a class that realises an interface must declare (or inherit from a superclass) all the interface operations. It is possible for a class to realise more than one interface, in which case the class must declare or inherit all the operations included in all the interfaces. Of course, a class may contain additional operations too.

Figure 17 shows how realisation is represented in UML. Notation (a) is used when there is an explicit definition of an interface; notation (b) is a shorthand version and only requires the interface name.

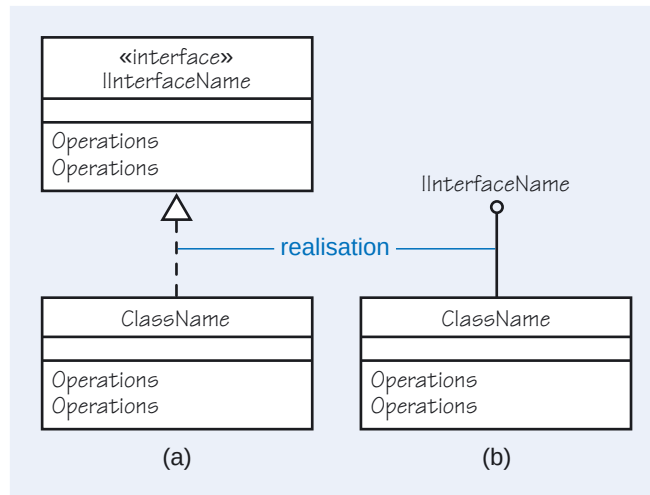


Figure 17 Alternative UML notations for the realisation of interfaces

The use of an interface by a class is indicated by a stereotyped dependency arrow, as shown in Figure 18.

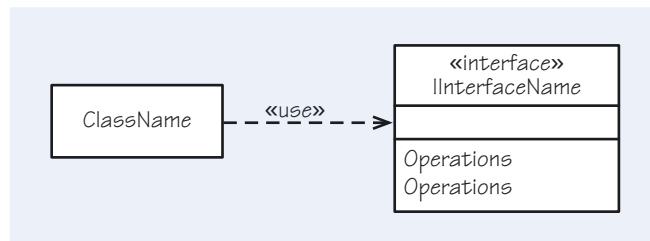


Figure 18 A class using an interface

A simple example of a possible component specification in UML is given in Figure 19, which shows a catalogue component offering a number of services, including some support for browsing the catalogue and searching for particular products.

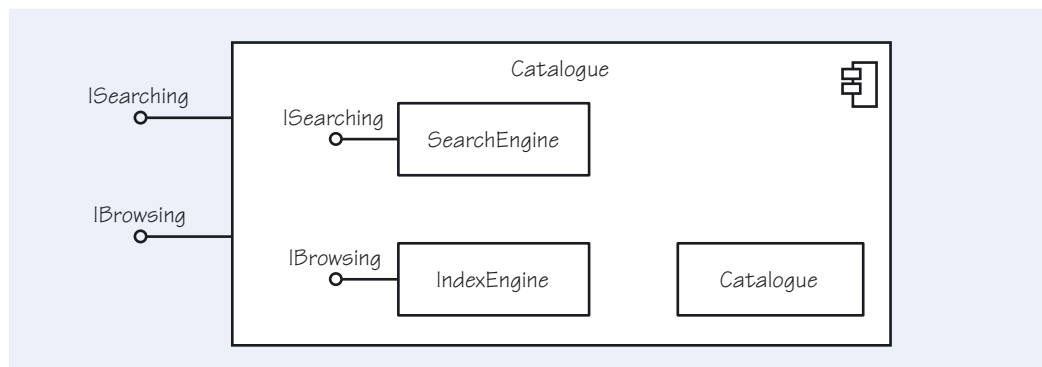


Figure 19 An example of a component specification

SAQ 10

To complete the specification of an interface, it is necessary to specify the operations within each interface. These operations will have pre- and postconditions. Suggest a technique to specify these conditions.

ANSWER.....

As with other types of operations you have encountered on the course, an interface operation could be specified by a contract written in OCL. This would specify the operation's context, its signature, and the pre- and postconditions. Alternatively, you could express the contract in natural language.

Exercise 3

Using the example in Figure 19, suggest a possible specification of the component's interfaces. Assume the existence of a type called `Collection` that permits the return of many items.

Solution

For browsing, a client may be able to browse alphabetically or by category of product. For searching, a client may be able to search by entering a keyword. A possible specification in UML is shown below.

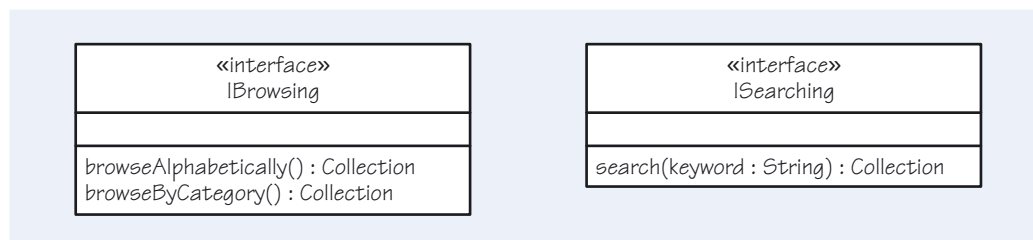


Figure 20 A possible specification

4.2 Summary of section

This section introduced components, and the UML notation used to describe them. This notation includes realisation of interfaces and component interconnection.



5

Summary

In this unit you extended your study of Design by Contract (DbC) from the assertions used to capture the specification of a system (introduced in *Unit 6*) to the later phases of development, particularly implementation. Java assertions were used to illustrate the ideas involved.

You learnt that being able to use assertions in code has a number of benefits that contribute to quality. At one level, they serve to document classes in terms of their contractual obligations to their clients and hence facilitate the correct use and reuse of such classes. At another level, if the assertions are executable, they ensure that a program is correct in terms of the specified contracts; that is, clients use suppliers correctly and suppliers deliver the correct results.

You also studied briefly how the process of inheriting assertions (in effect, inheriting contractual obligations) can be used to ensure that substitutability and allowing inherited methods to be redefined do not undermine the subtyping relationship.

UML provides diagramming techniques (component diagrams) to model the connections and dependencies between components. You have now encountered the full range of UML diagramming techniques covered by this course and are ready to consider some of the wider issues involved in implementing systems.

LEARNING OUTCOMES

On completion of this unit you should be able to:

- ▶ use Design by Contract (DbC) to help ensure design requirements are reflected in the implementation;
- ▶ understand some details of Java assertions;
- ▶ explain how being able to redefine inherited methods can undermine the subtyping relationship and cause problems in a language that supports substitutability;
- ▶ describe components and identify the parts of a component diagram.

Index

A

assertion (handling) mechanism 8

assertion mechanism 8

assertion statements 8

assertions 7

 executing 7

 language for 11

 making 7

C

class invariants 12

client code 15

components 25

components represented in UML 26

constructors 12

contracts 7

D

Design by Contract 6

documentation 13

E

encoding of requirements as assertions 6

executable assertions 13

executing an assertion using Java 7

expected cases 16

I

implementing the postcondition 9

inertial convention 11, 17

inheritance 20

interfaces (use of) 28

invariants 12

J

Java assertions 8

P

pre- and postconditions 7, 12

provided interface 26

R

realisation 28

required interface 26

requirements represented as assertions 11

S

software components 25

subcontracting 23

substitutability 19

supplier code 13

T

traceability 6

